

Python for Bioinformatics — Summing Values from the Command Line

What this program does

This small Python script lets you sum any number of numeric values you type after the program's name in the terminal.

For example:

```
python3 sum_values.py 10 2.5 7
```

Output:

```
values = [10.0, 2.5, 7.0]
sum = 19.5
```

This can be useful when working with read counts, expression values, or any numeric results you want to quickly add up.

The Complete Script

Save the following code as `sum_values.py`:

```
#!/usr/bin/env python3
import sys

def main():
    if len(sys.argv) < 2:
        sys.stderr.write(f"USAGE: python3 {sys.argv[0]} <value1> <value2> ... <valueN>\n")
        sys.exit(1)

    values = sys.argv[1:]

    try:
        # Convert all command-line inputs to floats (numbers with decimals)
        values = [float(v) for v in values]
    except ValueError as e:
        sys.stderr.write("Error: All input values must be numeric.\n")
        sys.exit(1)

    print(f"values = {values}")
    print(f"sum = {sum(values)}")

if __name__ == "__main__":
    main()
```

Step-by-Step Explanation

1) The shebang line

```
#!/usr/bin/env python3
```

This line tells your computer: “Run this file using Python 3.”

It is helpful if you want to run the script directly on macOS or Linux.

2) Importing a Python module

```
import sys
```

We bring in Python’s built-in `sys` module, which lets our program: - Access what was typed on the command line, - Show helpful error messages, and - Exit the program if something goes wrong.

3) Defining a main function

```
def main():
```

We define a function called `main()` — like a container for our steps. When Python runs this file, it will start from here.

4) Checking that the user typed some numbers

```
if len(sys.argv) < 2:
    sys.stderr.write(f"USAGE: python3 {sys.argv[0]} <value1> <value2> ... <valueN>\n")
    sys.exit(1)
```

- `sys.argv` is a list of all the words you typed when running the program.
Example:

```
python3 sum_values.py 5 10 15
```

```
gives sys.argv = ['sum_values.py', '5', '10', '15'].
```

- If you forget to type numbers, `len(sys.argv)` will be less than 2, so:
 - The script prints a usage message telling you what to do.
 - Then it exits with an error (code 1).

Bioinformatics analogy: Like a command-line tool that refuses to run until you specify your input FASTQ file.

5) Getting the input values

```
values = sys.argv[1:]
```

This collects everything after the script name. If you typed `python3 sum_values.py 10 2.5 7`, then `values` becomes:

```
["10", "2.5", "7"]
```

Notice these are still strings (text), not numbers yet!

6) Converting strings to numbers

```
try:
    values = [float(v) for v in values]
except ValueError as e:
    sys.stderr.write("Error: All input values must be numeric.\n")
    sys.exit(1)
```

- `[float(v) for v in values]` is a list comprehension — it loops through the list and converts each item into a `float` (a number with decimals).
- If any value cannot be converted (like “apple”), Python raises a `ValueError`.
- The `except` block catches that and shows a clear error message.

Bioinformatics analogy: Like converting a column of numeric gene counts from a CSV file before analysis.

7) Printing the results

```
print(f"values = {values}")
print(f"sum = {sum(values)}")
```

- The first line prints the numeric list (for a sanity check).
- The second line calculates and prints the sum using Python’s built-in `sum()` function.

Example output:

```
values = [10.0, 2.5, 7.0]
sum = 19.5
```

8) Telling Python when to start

```
if __name__ == "__main__":
    main()
```

This line means: “If this file is being run directly, start by calling `main()`.”

This is a Python best practice so you can reuse the script later as part of another project.

Why the Script Uses These Features

Feature	Reason
<code>float</code> instead of <code>int</code>	To handle decimals (e.g., TPM, RPKM, or normalized values).
<code>ValueError</code>	More precise than catching all errors.
<code>stderr</code> for errors	Keeps error messages separate from real results.
<code>main()</code> function	Makes the script reusable and organized.

How to Use It

Example 1 — Summing read counts

```
python3 sum_values.py 10342 9980 11021
```

Output:

```
values = [10342.0, 9980.0, 11021.0]
sum = 31343.0
```

Example 2 — Summing expression values

```
python3 sum_values.py 12.4 0.0 3.6 8.1
```

Output:

```
values = [12.4, 0.0, 3.6, 8.1]
sum = 24.1
```

Common Mistakes and Fixes

Problem	What Happens	How to Fix
You forget to type numbers	Usage message appears	Add numbers after the script name
You type text instead of a number	Error: All input values must be numeric	Remove text or use numeric-only input
You use commas	ValueError	Remove commas (e.g., use 1234 instead of 1,234)

Extensions (Optional Ideas)

If you want to go further: - Read numbers from a file (e.g., one number per line). - Accept input through a pipe: `bash cat counts.txt | python3 sum_values.py` - - Skip or ignore missing values like “NA”.

End of Tutorial